Simple Socket Programming

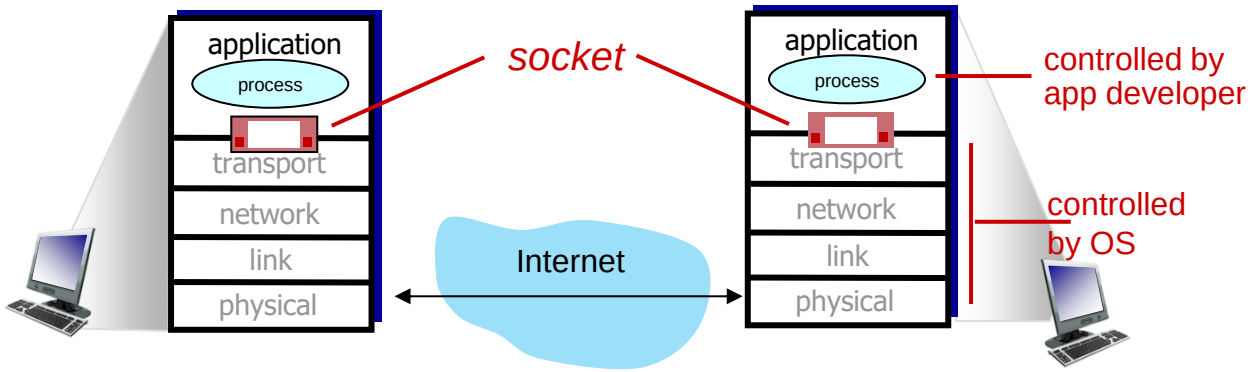
- Socket introduction
- Practice: Socket Programming in TCP
- Practice: Socket Programming in UDP
- Class Assignment: simple calculator with TCP socket

Index

- Background
  - Socket introduction
- Practice: Socket Programming for Echo Server & Client
  - Practice: Socket Programming in TCP with Blank
  - Practice: Socket Programming in UDP with Blank
- Class Assignment 1: simple calculator with TCP Socket
  - Overview: simple calculator with TCP Socket
  - Codes: Blank code for step-by-step practice
  - Results: Screenshots and Wireshark capture for result
- Appendix

Background: Sockets

- door between application process and end-end-transport protocol
- process sends/receives messages to/from its socket
- socket analogous to door
  - sending process shoves message out door
  - sending process relies on transport infrastructure on other side of door to deliver message to socket at receiving process
  - two sockets involved: one on each side



Practice: Socket Programming

- Goal:
  - learn how to build client/server application that communicate using sockets
  - Learn What is Socket and what is socket programming.
    - Socket is interface! And TCP/UDP is protocol!

- Socket API
- introduced in BSD4.1 UNIX, 1981
- explicitly created, used, released by apps
- client/server paradigm
- two types of transport service via socket API:
  - UDP: unreliable datagram
  - TCP: reliable, byte stream-oriented

socket

a *host-local*, *application-created*, *OS-controlled* interface (a "door") into which application process can both send and receive messages to/from another application process

Practice: Socket programming with TCP

Client must contact server

- server process must first be running
- server must have created socket (door) that welcomes client's contact

Client contacts server by:

- creating client-local TCP socket
- specifying IP address, port number of server process
- When client creates socket: client TCP establishes connection to server TCP

- When contacted by client, server TCP creates new socket for server process to communicate with client
  - allows server to talk with multiple clients
  - source port numbers used to distinguish clients (more in Chap 3)

application viewpoint

TCP provides reliable, in-order transfer of bytes ("pipe") between client and server

Practice: Socket programming with TCP

TCP service: reliable transfer of **bytes** from one process to another

server (running on hostid)

client

create socket, port=**x**,
For incoming request:

C : int sock = socket()
Python : serverSocket = socket()

welcomeSocket = ServerSocket()

wait for incoming connection request
connectionSocket = welcomeSocket.accept()

TCP connection setup

create socket, connect to **hostid**, port=**x**
clientSocket =Socket()

send request using clientSocket

read request from connectionSocket

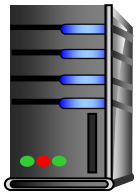
write reply to connectionSocket

read reply from clientSocket

close connectionSocket

close clientSocket

Practice: Sample Java TCP Server Code



server (running on hostid)

client

create socket,
port=**x**,
For incoming
request:

wait for incoming
connection request

read request from
connectionSocket

write reply to
connectionSocket

```
// Create server socket on port x

// Accept connection from client
Socket connectionSocket
                = welcomeSocket.accept();

// Create input stream to receive data from client

// Read a line of data from client
clientSentence = inFromClient.readLine();

// Send converted data back to client
outToClient.writeBytes(capitalizedSentence)
;
```

TCP
connection setup

create socket,
connect to **hostid**, port=**x**

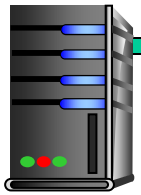
clientSocket =Socket()

send request using
clientSocket

read reply from
clientSocket

Close clientSocket

Practice: Sample Java TCP Client Code



server

(running on hostid)

create socket,
port=**x**,
For incoming
request:

wait for incoming
connection request

read request from
connectionSocket

write reply to
connectionSocket

TCP
connection setup

```
import java.io.*;
import java.net.*;

// Create socket to connect to server (ip, x)
Socket clientSocket = new Socket(ip,x);

new                    Reader(System.in));

// Create output stream to send data to server

// Read a line of input from user
sentence = inFromUser.readLine();
outToServer.writeBytes(sentence + '\n');

modifiedSentence = inFromServer.readLine();

System.out.println("FROM SERVER: "
    + modifiedSentence);
// Close the socket to end the connection
clientSocket.close();
}
}
```

client

create socket,
connect to **hostid**, port=**x**

clientSocket =Socket()

send request using
clientSocket

ead reply from
ientSocket

Close clientSocket

Practice: Socket Programming: TCP Result

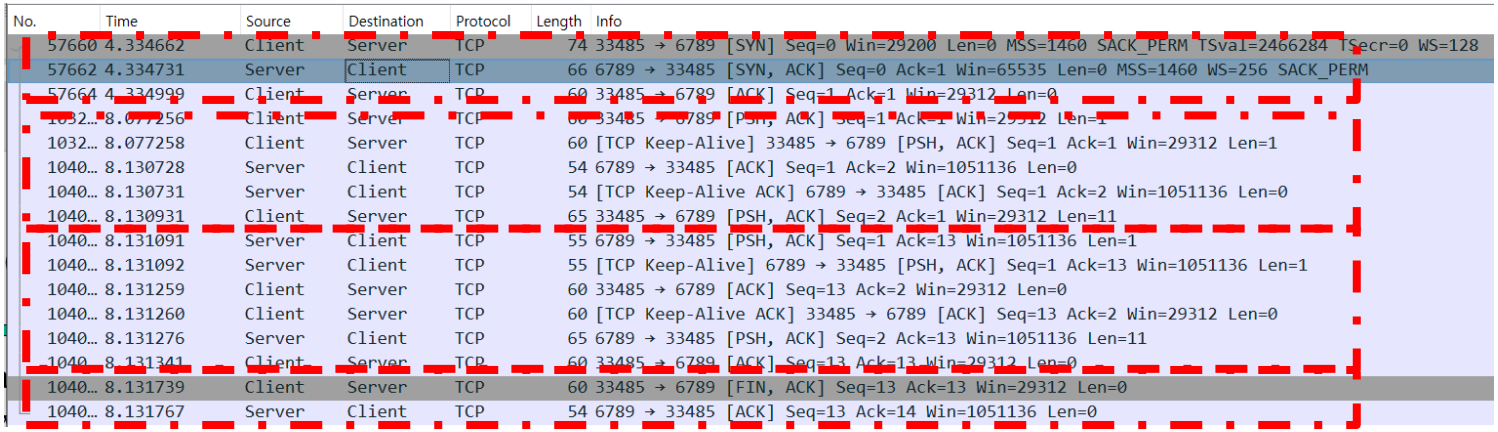
- Start Server Program First
  - Open Socket and wait
- execute Client Program
- Client exits after receiving message from server

① hand shake
(TCP connection setup)

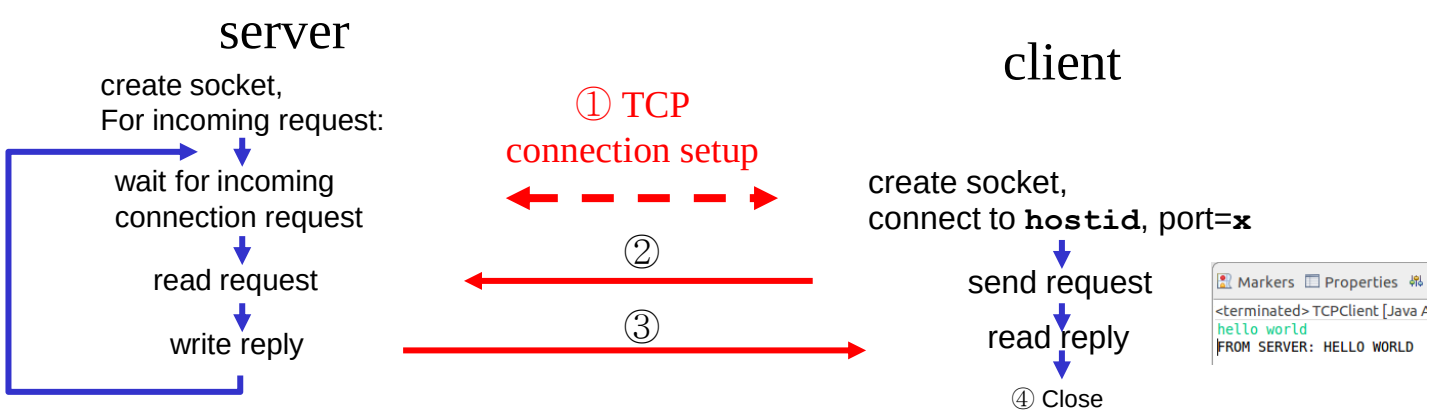
② send from client
(hello world)

③ reply to client
(HELLO WORLD)

④ close (FIN,ACK)



No.	Time	Source	Destination	Protocol	Length	Info
57660	4.334662	Client	Server	TCP	74	33485 → 6789 [SYN] Seq=0 Win=29200 Len=0 MSS=1460 SACK_PERM TSval=2466284 TSecr=0 WS=128
57662	4.334731	Server	Client	TCP	66	6789 → 33485 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
57664	4.334999	Client	Server	TCP	60	33485 → 6789 [ACK] Seq=1 Ack=1 Win=29312 Len=0
1032…	8.077256	Client	Server	TCP	60	33485 → 6789 [PSH, ACK] Seq=1 Ack=1 Win=29312 Len=1
1032…	8.077258	Client	Server	TCP	60	[TCP Keep-Alive] 33485 → 6789 [PSH, ACK] Seq=1 Ack=1 Win=29312 Len=1
1040…	8.130728	Server	Client	TCP	54	6789 → 33485 [ACK] Seq=1 Ack=2 Win=1051136 Len=0
1040…	8.130731	Server	Client	TCP	54	[TCP Keep-Alive ACK] 6789 → 33485 [ACK] Seq=1 Ack=2 Win=1051136 Len=0
1040…	8.130931	Client	Server	TCP	65	33485 → 6789 [PSH, ACK] Seq=2 Ack=1 Win=29312 Len=11
1040…	8.131091	Server	Client	TCP	55	6789 → 33485 [PSH, ACK] Seq=1 Ack=13 Win=1051136 Len=1
1040…	8.131092	Server	Client	TCP	55	[TCP Keep-Alive] 6789 → 33485 [PSH, ACK] Seq=1 Ack=13 Win=1051136 Len=1
1040…	8.131259	Client	Server	TCP	60	33485 → 6789 [ACK] Seq=13 Ack=2 Win=29312 Len=0
1040…	8.131260	Client	Server	TCP	60	[TCP Keep-Alive ACK] 33485 → 6789 [ACK] Seq=13 Ack=2 Win=29312 Len=0
1040…	8.131276	Server	Client	TCP	65	6789 → 33485 [PSH, ACK] Seq=2 Ack=13 Win=1051136 Len=11
1040…	8.131341	Client	Server	TCP	60	33485 → 6789 [ACK] Seq=13 Ack=13 Win=29312 Len=0
1040…	8.131739	Client	Server	TCP	60	33485 → 6789 [FIN, ACK] Seq=13 Ack=13 Win=29312 Len=0
1040…	8.131767	Server	Client	TCP	54	6789 → 33485 [ACK] Seq=13 Ack=14 Win=1051136 Len=0



Practice: Socket programming with UDP

UDP: no "connection" between client and server

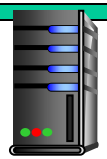
- no handshaking
- sender explicitly attaches IP address and port of destination to each packet
- server must extract IP address, port of sender from received packet

UDP: transmitted data may be received out of order, or lost

application viewpoint

UDP provides <u>unreliable</u> transfer of groups of bytes ("datagrams") between client and server

Practice: Socket programming with UDP



server (running on hostid)

create socket, port=**x**,
for incoming request:

serverSocket = DatagramSocket()

C : int sock = socket()
Python : serverSocket = socket()

client

create socket,

clientSocket = DatagramSocket()

Create diagram,
address (**hostid,port=x**),
send datagram using
clientSocket

read request from
serverSocket

write reply to serverSocket
specifying client
host address, port number

read reply from
clientSocket

Close
clientSocket

Practice: Sample Java UDP Server Code

server
(running
on hostid)

import
import

class U
pub

// Createerver socket on port x
DatagramSocket serverSocket =
new DatagramSocket(x)

client

create socket

create socket,
port=**x**,
For incoming
request:

// Receive packet from client
serverSocket.receive(receivePacket);

Create diagram
with address
send datagram
using clientSocket

```
// Convert received data to a string
String sentence = new String(receivePacket.getData());

// Get the client's IP address and port number
InetAddress IPAddress = receivePacket.getAddress();
int port = receivePacket.getPort();
```

read request from
serverSocket

// Send packet to client
serverSocket.send(sendPacket);

write reply to
serverSocket
specifying client
host address, port
number

}
}
}

read reply from
clientSocket

Close clientSocket

Practice: Sample Java UDP Client Code

server
(running
on hostid)

create socket,
port=**x**,
For incoming
request:

read request from
serverSocket

write reply to
serverSocket
specifying client
host address, port
number

ent

create socket

```
// Create UDP socket
DatagramSocket clientSocket
= new DatagramSocket();
```

```
// Create a packet to send
DatagramPacket sendPacket
    = new DatagramPacket
      (sendData, sendData.length, IPAddress, x);
// Send the packet to the server
```

```
// Receive the packet from the server
clientSocket.receive(receivePacket);
```

```
// Close the socket to end the connection
clientSocket.close();
```

ad reply from
clientSocket

Close clientSocket

Practice: Socket Programming: UDP Result

- Start Server Program First
  - Open Socket and wait
- After turn on Server program and execute Client Program
- Client exits after receiving message from server

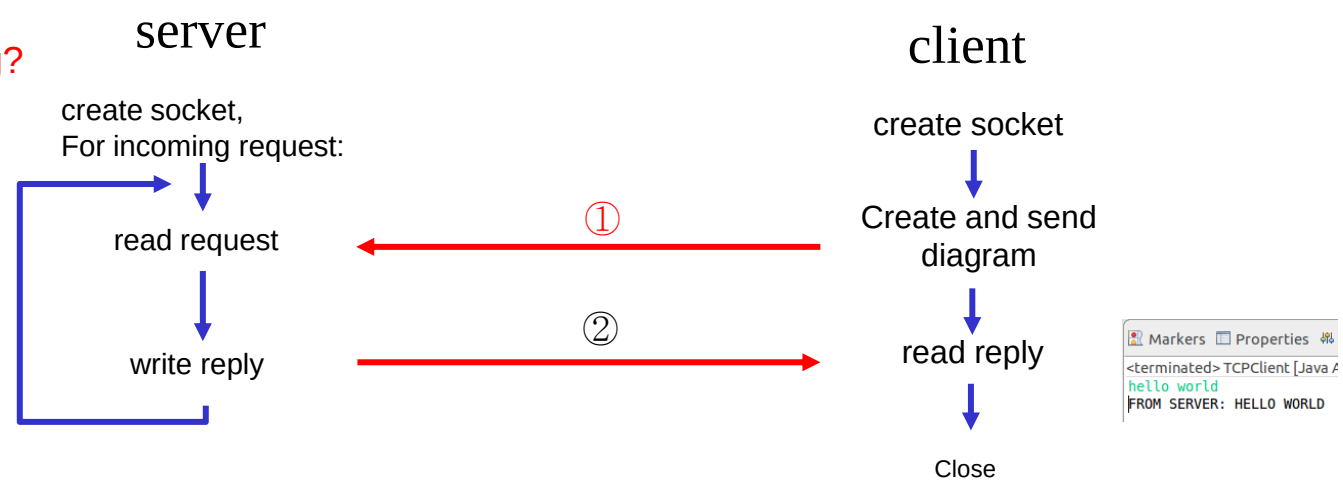
① send from client
(hello world)

② reply to client
(HELLO WORLD)

No.	Time	Source	Destination	Protocol	Length	Info
48550	4.705394	Client	Server	UDP	60	58103 → 9876 Len=11
48551	4.705396	Client	Server	UDP	60	58103 → 9876 Len=11
48556	4.705919	Server	Client	UDP	1066	9876 → 58103 Len=1024
48557	4.705922	Server	Client	UDP	1066	9876 → 58103 Len=1024

→ why length is so long?

→ In the Server byte[] sendData = new byte[1024];



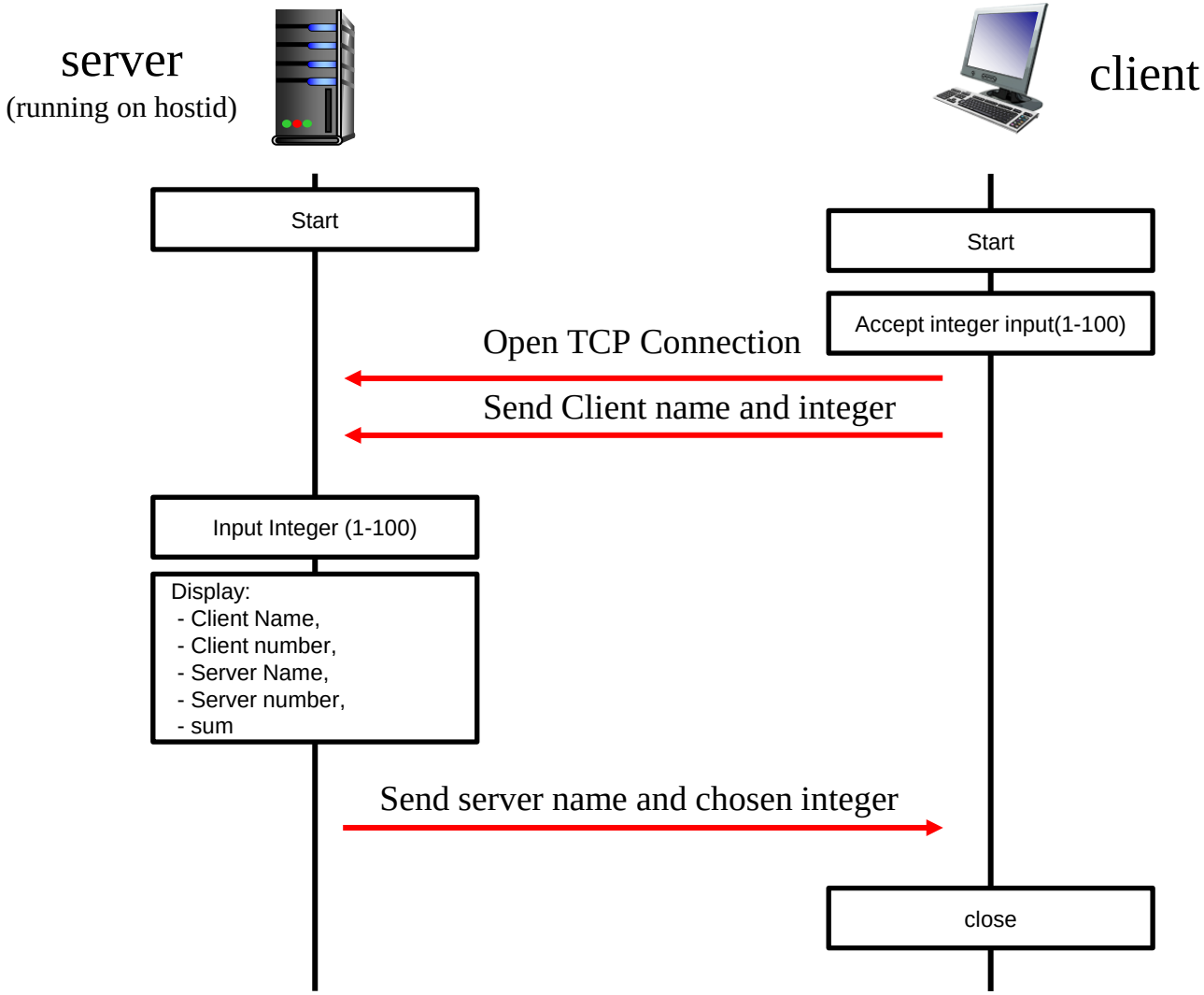
Extra Notes

- Q1. What is different with TCP&UDP?
  - Speed, reliability, stream(vs datagram), and else?

Class Assignment 1: Socket Programming: Overview(Simple Calculator) (1/2)

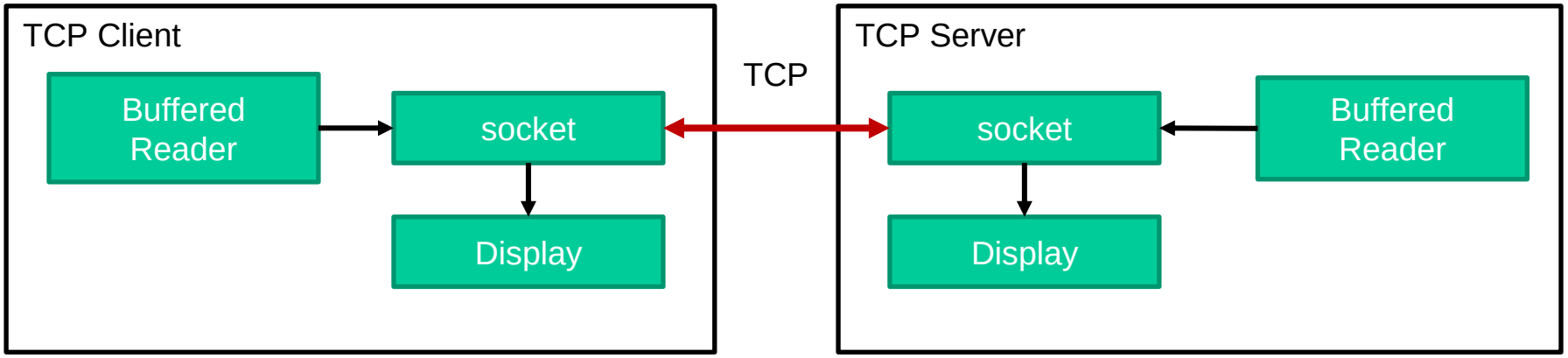
- Goal: write a client and server that will use sockets to communicate with.
  - Client:
    - should first accept an integer between 1 and 100 from the keyboard, open a TCP socket to your server and send a message containing
    - (i) a string containing your name (e.g., “Client of John Q. Smith”) and (ii) the entered integer value and then wait for a server reply.
  - server:
    - will create a string containing its name (e.g., “Server of John Q. Smith”) and then begin accepting connections from clients. On receipt of a client message, your server should
    - print (display) the client’s name (extracted from the received message) and the server’s name
    - itself pick an integer between 1 and 100 (it’s fine for the server to use the same number all the time) and display the client’s number, its number, and the sum of those numbers
    - send its name string and the server-chosen integer value back to the client
    - if your server receives an integer value that is out of range, it should terminate after releasing any created sockets. You can use this to shut down your server.

Class Assignment 1: Socket Programming Overview(Simple Calculator) (2/2)



What To Do(Mission 0)

- Before follow following the missions:
  - Draw Block diagram of TCP Client and TCP Server
  - Draw Flow chart for TCP Client and TCP Server

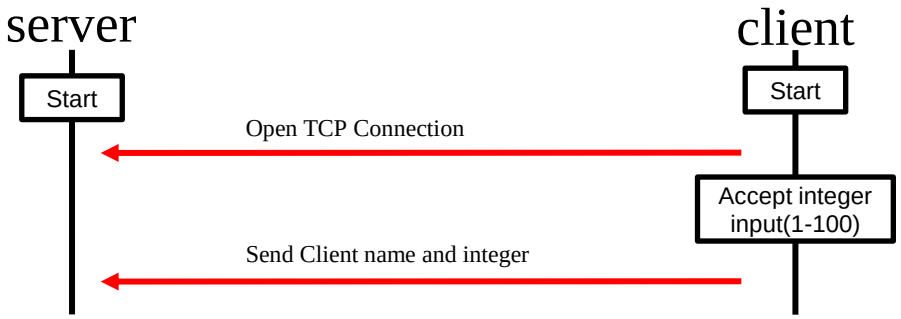


What To Do: Missions for programming

- Mission 1: Open TCP Socket to connect with Server and send a message
  - Fill#1, Initializing client-side socket to connect to Server
  - Fill#2, Read an integer from the user to make a message to send
  - Fill#3, Send the client name and the entered integer to the server
- Mission 2: Open TCP Socket to receive message from client
  - Fill#4, Initializing the server on a specified port
  - Fill#5, Listen for a TCP connection request.
  - Fill#6, Read message from client
- Mission 3: Extract client's name and integer from message and send calculated message to client
  - Fill#7, Calculate and print the sum of the client's and server's numbers
  - Fill#8, Send server's name and chosen number back to the client
- Mission 4: Extract the server's chosen integer from the response and Calculate the sum of the client’s and server’s numbers
  - Fill#8, Read and print the response from the server
  - Fill#9, Calculate and print the sum of the client's and server's numbers
  - Fill#10, Close socket

Class Assignment 1: Blank Code for Client

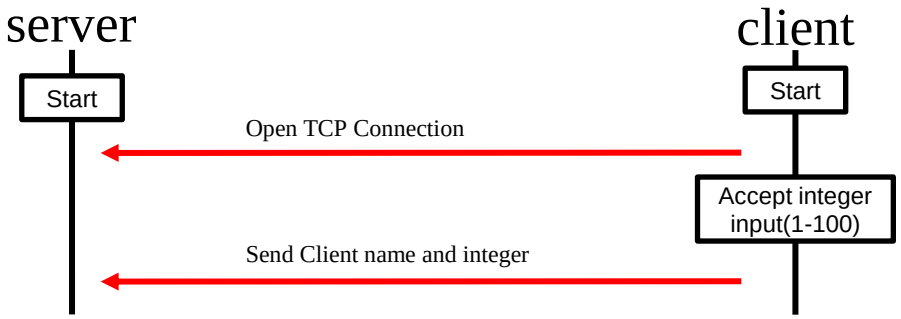
Mission 1: Open TCP Socket to connect with Server and send a message




```
import java.io.*;
import java.net.*;

c
//Fill#1, Initializing client-side socket to connect to Server
Socket clientSocket
 = new Socket(host, port);

// Fill#2, Read an integer from the user to make a message to send
System.out.print("Enter an integer between 1 and 100: ");
int userNumber = Integer.parseInt(inFromUser.readLine());
// Fill#3, Send the client name and the entered integer to the
server
String clientMessage = "Client Name: " + userNumber;
outToServer.writeBytes(clientMessage + '\n');
```

Class Assignment 1: Blank Code for Server

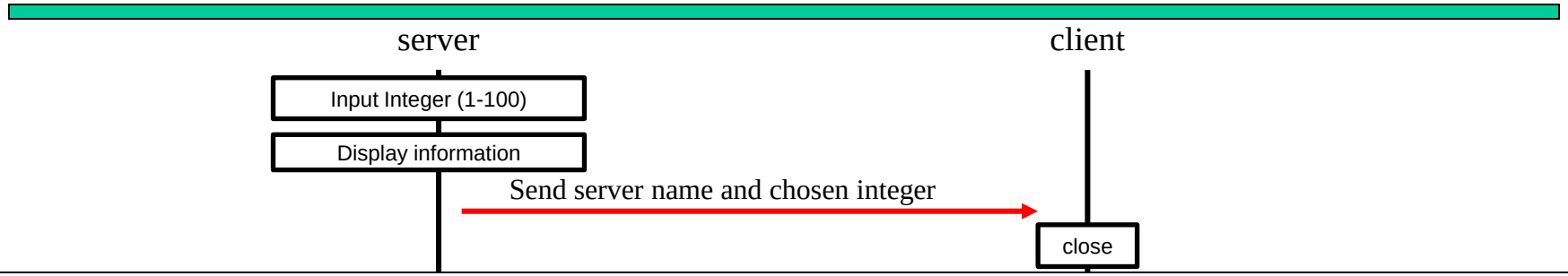
Mission 2: Open TCP Socket to receive message from client




```
import java.io.*;

// Fill#4, Initializing the server on a specified port
ServerSocket welcomeSocket = new ServerSocket(port);

// Fill#5, Listen for a TCP connection request.
Socket connectionSocket = welcomeSocket.accept();

whil

// Fill#6, Read message from client
String clientMessage = inFromClient.readLine();
System.out.println("Received: " + clientMessage);
```

Class Assignment 1: Blank Code for Server

Mission 3: Extract client's name and integer from message and send calculated message to client

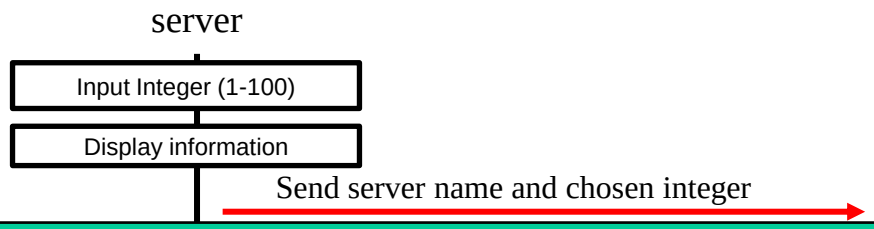



```
            // Mission 3: Extract client's name and integer from message and send calculated message to client
            String[] parts = clientMessage.split(": ");

            // Fill#7, Calculate and print the sum of the client's and server's numbers
            System.out.println("Client Name: " + clientName + "Client number: " + clientNumber);
            System.out.println("Server Name: " + serverName + "Server number: " + serverNumber);
            System.out.println("Sum: " + clientNumber + serverNumber);
            // Fill#8, [illegible] name and chosen number back to the client
            String serverMessage = serverName + ": " + serverNumber;
            [illegible]);
            [illegible]

            // [illegible] the server if the client sends a number outside the range of 1 to 100
            if (clientNumber < 1 || clientNumber > 100) {
                connectionSocket.close();
                welcomeSocket.close();
                break;
            }
        }
    }
}
```

For what?
Is this right Place? → I don't think so..

Class Assignment 1: Blank Code for Client

Mission 4: Extract the server's chosen integer from the response and Calculate the sum of the client's and server's numbers




```
        // Fill#8, Read and print the response from the server
        String serverMessage = inFromServer.readLine();
        System.out.println("FROM SERVER: " + serverMessage);
client's
        // Fill#8, Read and
        // Fill#9, Calculate and print the sum of the client's and server's
        numbers
        System.out.println("Client number: " + userNumber);
        System.out.println("Server number: " + serverNumber);
        System.out.println("Sum: " + userNumber + serverNumber);
        // Fill#10, Close socket
        clientSocket.close();
        // Fill#10,
    }
}
```

Class Assignment 1: Socket Programming: Results

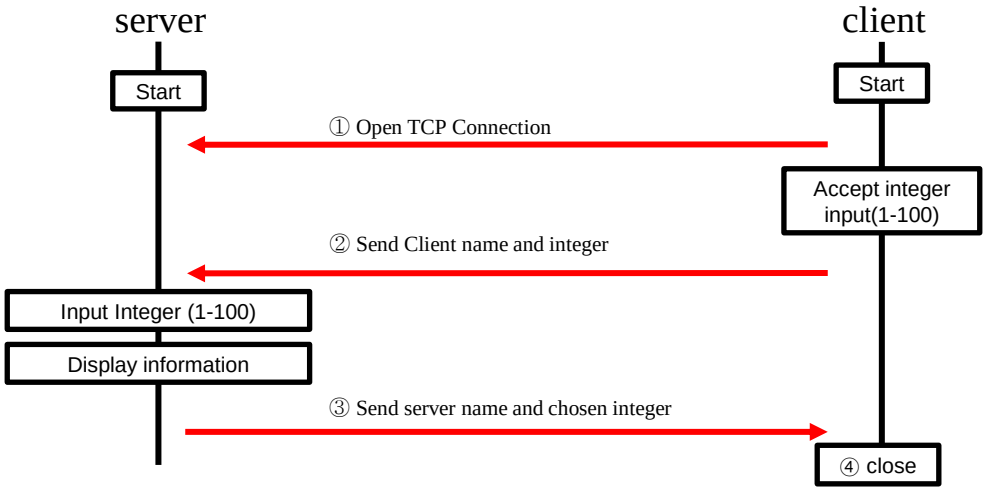
① Open TCP Connection

② Send from Client
(name and integer)

③ Send from server
(name and integer)

④ close (FIN,ACK)

Source	Destination	Protocol	Length	Info
Client	Server	TCP	74	34622 → 8888 [SYN] Seq=0 Win=29200 Len=0 MSS=1460 SACK_PERM TSval=418599 TSecr=0 WS=128
Server	Client	TCP	66	8888 → 34622 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
Client	Server	TCP	60	34622 → 8888 [ACK] Seq=1 Ack=1 Win=29312 Len=0
Client	Server	TCP	60	34622 → 8888 [PSH, ACK] Seq=1 Ack=1 Win=29312 Len=1
Client	Server	TCP	60	[TCP Keep-Alive] 34622 → 8888 [PSH, ACK] Seq=1 Ack=1 Win=29312 Len=1
Server	Client	TCP	54	8888 → 34622 [ACK] Seq=1 Ack=2 Win=1051136 Len=0
Server	Client	TCP	54	[TCP Keep-Alive ACK] 8888 → 34622 [ACK] Seq=1 Ack=2 Win=1051136 Len=0
Client	Server	TCP	81	34622 → 8888 [PSH, ACK] Seq=2 Ack=1 Win=29312 Len=27
Server	Client	TCP	54	8888 → 34622 [ACK] Seq=1 Ack=29 Win=1051136 Len=0
Server	Client	TCP	55	8888 → 34622 [PSH, ACK] Seq=1 Ack=29 Win=1051136 Len=1
Server	Client	TCP	55	[TCP Keep-Alive] 8888 → 34622 [PSH, ACK] Seq=1 Ack=29 Win=1051136 Len=1
Client	Server	TCP	60	34622 → 8888 [ACK] Seq=29 Ack=2 Win=29312 Len=0
Client	Server	TCP	60	[TCP Keep-Alive ACK] 34622 → 8888 [ACK] Seq=29 Ack=2 Win=29312 Len=0
Server	Client	TCP	81	8888 → 34622 [PSH, ACK] Seq=2 Ack=29 Win=1051136 Len=27
Client	Server	TCP	60	34622 → 8888 [ACK] Seq=29 Ack=29 Win=29312 Len=0
Client	Server	TCP	60	34622 → 8888 [FIN, ACK] Seq=29 Ack=29 Win=29312 Len=0
Server	Client	TCP	54	8888 → 34622 [ACK] Seq=29 Ack=30 Win=1051136 Len=0



Assignment:

- After the upon missions:
- Write report about assignment. report must have:
  - **Program Introduction**
    - Results of Mission 0
  - **Contents of the attachment**
    - Explanation of code with you own perspective
  - **Instructions: How to run the program**
  - **Wireshark**
    - Screenshot of result and wireshark
    - Explanation and analysis of packets through wireshark
  - **How the program works**

Extra Notes

- Complete this mission with UDP.
  - Is this program proper to build with UDP?
  - What is the Strong point? And what is the Weak point?

Appendix

Java TCP Calculator Full Code

Class Assignment 1: Full Code for Java Server

```java
import java.io.*;
import java.net.*;

class TCPClient {
    public static void main(String argv[]) throws Exception {
        String host = "localhost"; // Server address
        int port = 8888; // Server port number

        // Open TCP Connection
        Socket clientSocket = new Socket(host, port);

        BufferedReader inFromUser = new BufferedReader(new InputStreamReader(System.in));
        DataOutputStream outToServer = new DataOutputStream(clientSocket.getOutputStream());
        BufferedReader inFromServer = new BufferedReader(new InputStreamReader(clientSocket.getInputStream()));

        // Read an integer from the user
        System.out.print("Enter an integer between 1 and 100: ");
        int userNumber = Integer.parseInt(inFromUser.readLine());

        // Send the client's name and the entered integer to the server
        String clientMessage = "Client of John Q. Smith: " + userNumber;
        outToServer.writeBytes(clientMessage + '\n');

        // Read and print the response from the server
        String serverMessage = inFromServer.readLine();
        System.out.println("FROM SERVER: " + serverMessage);

        // Extract the server's chosen integer from the response
        String[] parts = serverMessage.split(": ");
        String serverName = parts[0];
        int serverNumber = Integer.parseInt(parts[1]);

        // Calculate and print the sum of the client's and server's numbers
        int sum = userNumber + serverNumber;
        System.out.println("Client number: " + userNumber);
        System.out.println("Server number: " + serverNumber);
        System.out.println("Sum: " + sum);

        // Close TCP Connection from client
        clientSocket.close();
    }
}
```

Class Assignment 1: Full Code for Java Client

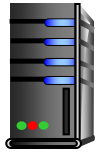
```java
import java.io.*;
import java.net.*;

class TCPServer {
    public static void main(String argv[]) throws Exception {
        String serverName = "Server of John Q. Smith"; // Server name
        int port = 8888; // Server port number

        ServerSocket welcomeSocket = new ServerSocket(port);

        while (true) {
            // Open TCP Connection
            Socket connectionSocket = welcomeSocket.accept();
            BufferedReader inFromServer = new BufferedReader(new InputStreamReader(System.in));
            BufferedReader inFromClient = new BufferedReader
                                        (new InputStreamReader(connectionSocket.getInputStream()));
            DataOutputStream outToClient = new DataOutputStream(connectionSocket.getOutputStream());

            // Read message from client
            String clientMessage = inFromClient.readLine();
            System.out.println("Received: " + clientMessage);

            // Extract client's name and integer
            String[] parts = clientMessage.split(": ");
            String clientName = parts[0];
            int clientNumber = Integer.parseInt(parts[1]);

            // Terminate the server if the client sends a number outside the range of 1 to 100
            if (clientNumber < 1 || clientNumber > 100) {
                connectionSocket.close();
                welcomeSocket.close();
                break;
            }

            // Server's chosen integer (between 1 and 100, always set to 50 here)
            System.out.print("Enter an integer between 1 and 100: ");
            int serverNumber = Integer.parseInt(inFromServer.readLine());

            // Calculate and print the sum of the client's and server's numbers
            System.out.println("Client Name: " + clientName + "Client number: " + clientNumber);
            System.out.println("Server Name: " + serverName + "Server number: " + serverNumber);
            System.out.println("Sum: " + clientNumber + serverNumber);

            // Send server's name and chosen number back to the client
            String serverMessage = serverName + ": " + serverNumber;
            outToClient.writeBytes(serverMessage + '\n');
        }
    }
}
```

Appendix

Python TCP/UDP Sample Code

Simple Socket programming with TCP

TCP service: reliable transfer of **bytes** from one process to another



server (running on hostid)

client

create socket,
port=**x**, for incoming request:

C : int sock = socket()

serverSocket = socket() Java : sock = ServerSocket()

wait for incoming
connection request
connectionSocket =
serverSocket.accept()

TCP
connection setup

create socket,
connect to **hostid**, port=**x**
clientSocket = socket()

send request using
clientSocket

read request from
connectionSocket

write reply to
connectionSocket

read reply from
clientSocket

close
connectionSocket

close
clientSocket

Socket Program
TCP Server

- Server

```
import socket      // import needed package

HOST = '127.0.0.1'   // Declare IP of Server

PORT = 50000         // Declare Port of Server

BUFFER = 4096        // Declare Buffer Size of Server

sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM) // Type of Socket in Server Server  (Type of Protocol)

sock.bind((HOST, PORT)) // Bind of the Socket, IP/Port of Server (Server Setting)

sock.listen(0)          // Start to Listen in Server : Ready to take a client

print('tcpServer listen at: %s:%s\n\r' % (HOST, PORT))

while True:                                     // Whenever Connection access comes, accept it
    client_sock, client_addr = sock.accept()
    print('%s:%s connect' % client_addr)        // Print IP and Port of Client

    while True:

        recv = client_sock.recv(BUFFER)         // Store Massge text in Buffer which if come from Client

        if not recv:                            // Close Socket if recv is false

            client_sock.close()

            break

        print('[Client %s:%s said]:%s' % (client_addr[0], client_addr[1], recv))  // Print from buffer which come from Client

        client_sock.send('tcpServer has received your message')                   // Response to Client


sock.close()      // Socket 닫는다.
```

Socket Program
TCP Client

- Client

```
import socket   // import needed Package

HOST='127.0.0.1'  // Declare Server IP

PORT=50000        // Declare Server Port

BUFFER=4096       // Declare Buffer size

sock=socket.socket(socket.AF_INET,socket.SOCK_STREAM)        // Type of Socket of Server (Type of Protocol)

sock.connect((HOST,PORT))       // Connect to Server

sock.send('hello, tcpServer!') // Send Message to Server

recv=sock.recv(BUFFER)          // Store message in Buffer which is from Server

print('[tcpServer said]: %s' % recv)  // print context from Buffer

sock.close()                            // Close Socket
```

Socket Program
TCP

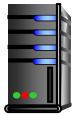
- Start Server Program First
  - Open Socket and wait
- After turn on Server program and execute Client Program
- Client exits after receiving message from server
- Client

```
Console
<terminated> __init__.py [C:₩Python27₩python.exe]
[tcpServer said]: tcpServer has received your message
```

- Server

```
Console
__init__.py [C:₩Python27₩python.exe]
tcp server listen at : 127.0.0.1 : 50000

127.0.0.1:1700 connected
[client 127.0.0.1 : 1700 said ] : hello, tcpServer!
```

Client/server socket interaction: UDP



server (running on serverIP)

create socket, port= x:

serverSocket = socket()

C : int sock = socket()
Java : serverSocket = Datagramsocket()

read datagram from serverSocket

write reply to serverSocket specifying client address, port number

client

create socket:

clientSocket = socket()

Create datagram with server IP and port=x; send datagram via clientSocket

read datagram from clientSocket

close clientSocket

Socket Program
UDP

- Server

```
import socket                                                    // import Package

HOST='127.0.0.1'                                                 // Server IP

PORT=50001                                                       // Server Port

BUFFER=4096                                                      // Server Buffer

sock=socket.socket(socket.AF_INET,socket.SOCK_DGRAM)             // Server Socket type (Protocol type)

sock.bind((HOST,PORT))                                           // Bind Server Socket IP/Port

#sock.listen(0)                                                  // do not need to real-time listen

print('UDPServer listen at: %s:%s\n\r' %(HOST,PORT))

while True:
                                                                 //  do not need to Accept
    #client_sock,client_addr=sock.accept()

    #print('%s:%s connect' %client_addr)

    while True:

        recv,client_addr=sock.recvfrom(BUFFER)
                                                                 // Save message context from client in Buffer
        if not recv:
                                                                 // close Socket when recv is false
            break

        print('[Client %s:%s said]:%s' % (client_addr[0],client_addr[1],recv))   //print the context from Client

        sock.sendto('UDPServer has received your message',client_addr)   // reply to Client

sock.close();                                                    // Close Socket
```

Socket Program
UDP

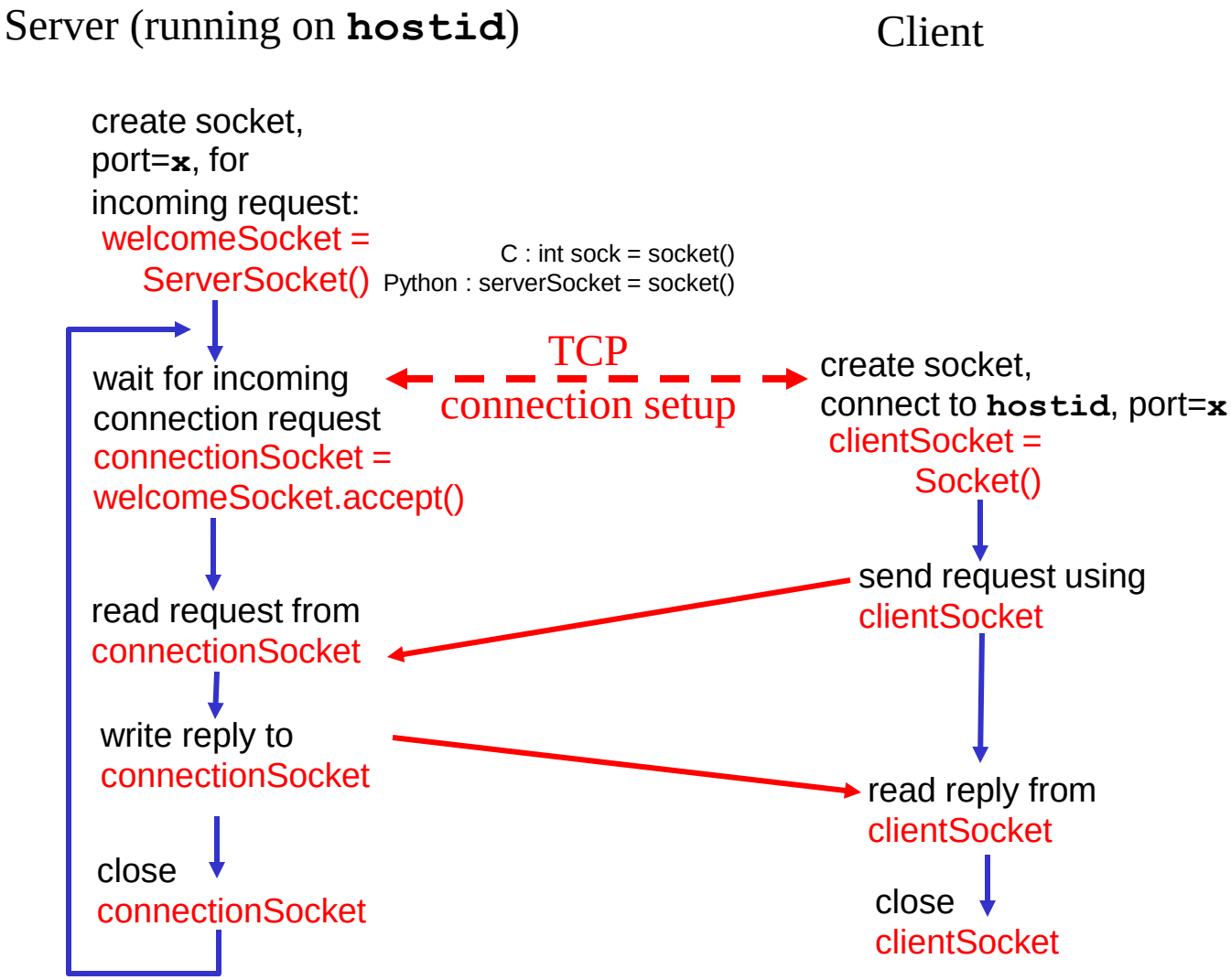
- Client

```
import socket                                   // import Package

HOST='127.0.0.1'                                // Server IP

PORT=50001                                      // Server Port

BUFFER=4096                                     // Buffer size

sock=socket.socket(socket.AF_INET,socket.SOCK_DGRAM)        // Server Socket type (Protocol type)

sock.connect((HOST,PORT))                       // Server Connection

sock.send('hello, UDPServer!')                  // Send Message to Server보낸다

recv=sock.recv(BUFFER)                          // Save message from the Server in buffer

print('[UDPServer said]: %s' % recv)            // Save context in Buffer

sock.close()                                    // Close Socket
```

Appendix

Java TCP/UDP Old Source Code

Socket programming with TCP



Socket programming with TCP
Ex _Server (1/2)

```
import java.io.*;
import java.net.*;

class TCPServer {
    public static void main(String argv[]) throws Exception
    {
      String clientSentence;
      String capitalizedSentence;
      ServerSocket welcomeSocket = new ServerSocket(6789);

      while(true) {

          Socket connectionSocket = welcomeSocket.accept();


          BufferedReader inFromClient =
             new BufferedReader(new
             InputStreamReader(connectionSocket.getInputStream()));

      }
```

Create welcoming socket at port 6789

Wait, on welcoming socket for contact by client

Create input stream, attached to socket

Socket programming with TCP
Ex _Server (2/2)

Create output stream, attached to socket

Read in line from socket

Write out line to socket

```
DataOutputStream  outToClient =
  new DataOutputStream(connectionSocket.getOutputStream());

clientSentence = inFromClient.readLine();

capitalizedSentence = clientSentence.toUpperCase() + '\n';

outToClient.writeBytes(capitalizedSentence);
            }
        }
    }
```

End of while loop, loop back and wait for another client connection

Socket programming with TCP
Ex _Client (1/2)

```
import java.io.*;
import java.net.*;

class TCPClient {

    public static void main(String argv[]) throws Exception
    {
        String sentence;
        String modifiedSentence;

        BufferedReader inFromUser =
          new BufferedReader(new InputStreamReader(System.in));

        Socket clientSocket = new Socket("hostname", 6789);

        DataOutputStream outToServer =
          new DataOutputStream(clientSocket.getOutputStream());
```

Create input stream

Create client socket, connect to server

Create output stream attached to socket

Socket programming with TCP
Ex _Client (2/2)

Create input stream attached to socket

Send line to server

Read line from server

```
BufferedReader inFromServer =
    new BufferedReader(new
    InputStreamReader(clientSocket.getInputStream()));

sentence = inFromUser.readLine();
outToServer.writeBytes(sentence + '\n');

modifiedSentence = inFromServer.readLine();
System.out.println("FROM SERVER: " + modifiedSentence);

clientSocket.close();

    }
}
```

Socket programming with UDP

Server (running on **`hostid`**)

Client

create socket,
port=**`x`**, for
incoming request:
serverSocket =
DatagramSocket()

C : int sock = socket()
Python : serverSocket = socket()

create socket,
clientSocket =
DatagramSocket()

Create, address (**`hostid, port=x,`**
send datagram request
using clientSocket

read request from
serverSocket

write reply to
serverSocket
specifying client
host address,
port number

read reply from
clientSocket

close
clientSocket

Socket programming with UDP
Ex _Server (1/2)

```
import java.io.*;
import java.net.*;

class UDPServer {
  public static void main(String args[]) throws Exception
    {

      DatagramSocket serverSocket = new DatagramSocket(9876);

      byte[] receiveData = new byte[1024];
      byte[] sendData  = new byte[1024];

      while(true)
        {

          DatagramPacket receivePacket =
             new DatagramPacket(receiveData, receiveData.length);
           serverSocket.receive(receivePacket);
```

Create datagram socket at port 9876

Create space for received datagram

Receive datagram

Socket programming with UDP
Ex _Server (2/2)

Get IP addr port #, of sender

Create datagram to send to client

Write out datagram to socket

```
String sentence = new String(receivePacket.getData());

InetAddress IPAddress = receivePacket.getAddress();

int port = receivePacket.getPort();

String capitalizedSentence = sentence.toUpperCase();

sendData = capitalizedSentence.getBytes();

DatagramPacket sendPacket =
   new DatagramPacket(sendData, sendData.length, IPAddress,
                      port);

serverSocket.send(sendPacket);
}
}
}
```

End of while loop, loop back and wait for another datagram

Socket programming with UDP
Ex _Client (1/2)

Create input stream

Create client socket

Translate hostname to IP address using DNS

```
import java.io.*;
import java.net.*;

class UDPClient {
    public static void main(String args[]) throws Exception
    {

      BufferedReader inFromUser =
        new BufferedReader(new InputStreamReader(System.in));

      DatagramSocket clientSocket = new DatagramSocket();

      InetAddress IPAddress = InetAddress.getByName("hostname");

      byte[] sendData = new byte[1024];
      byte[] receiveData = new byte[1024];

      String sentence = inFromUser.readLine();
      sendData = sentence.getBytes();
```

Socket programming with UDP
Ex _Client (2/2)

Create datagram with data-to-send, length, IP addr, port

Send datagram to server

Read datagram from server

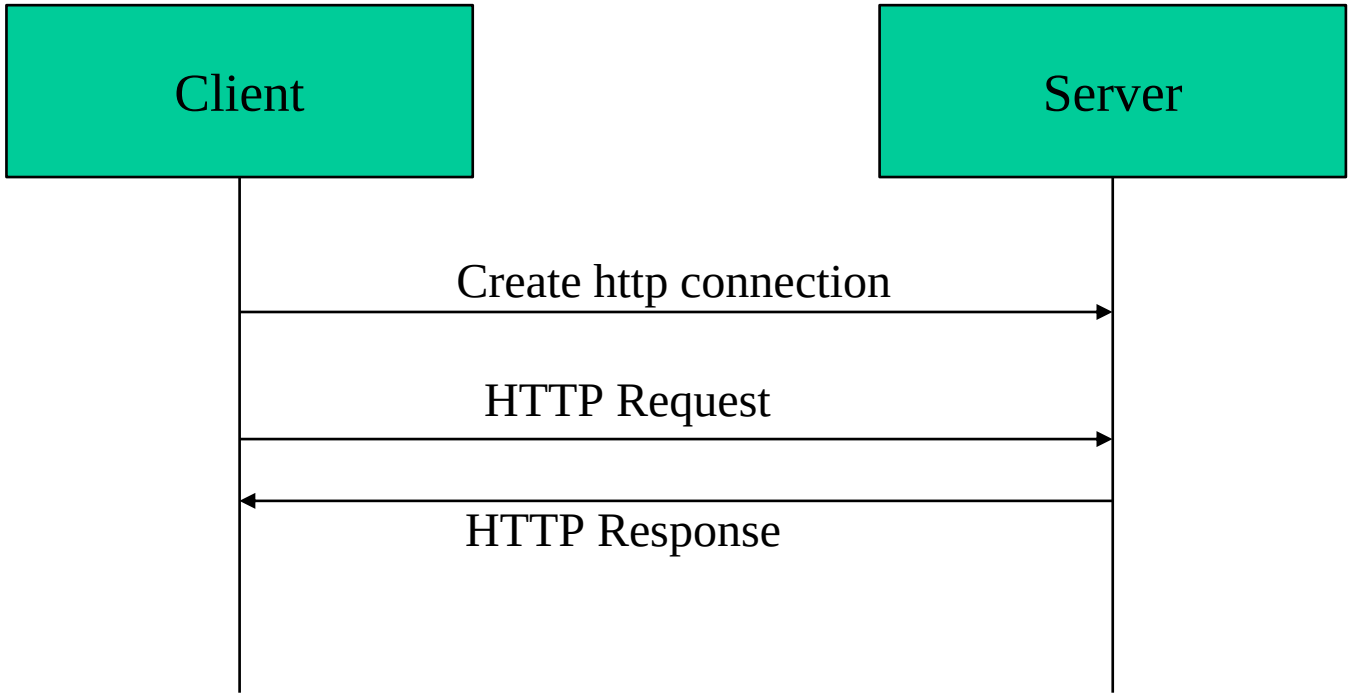
```
        DatagramSocket clientSocket = new DatagramSocket();

        InetAddress IPAddress = InetAddress.getByName("hostname");

        byte[] sendData = new byte[1024];
        byte[] receiveData = new byte[1024];

        String sentence = inFromUser.readLine();
        sendData = sentence.getBytes();
        DatagramPacket sendPacket = new DatagramPacket(sendData, sendData.length, IPAddress, 9876);

        clientSocket.send(sendPacket);

        DatagramPacket receivePacket = new DatagramPacket(receiveData, receiveData.length);

        clientSocket.receive(receivePacket);

        String modifiedSentence = new String(receivePacket.getData());

        System.out.println("FROM SERVER:" + modifiedSentence);
        clientSocket.close();
    }
}
```

Appendix

web Programming

Web Programming

- Based on HTTP (HyperText Transfer Protocol)
- Web-Client connects with Web-Server in HTTP



HTTP Request & Response

- HTTP Request
  - Get
  - Post
  - Put
  - Head
  - ….
- HTTP Response
  - 200 OK
  - 301 Moved Permanently
  - 400 Bad Request
  - 404 Not Found
  - 505 HTTP Version Not Supported
  - ….